

OOPJ — Day 12

Sunday, 20 September 2026 | Cursors, List Implementations, Set, Map & Sorting

1. Cursors in Java

A **cursor** in Java is a mechanism that allows you to retrieve elements one by one from a collection object. Java provides three cursor interfaces, each with different capabilities and applicable contexts.

Enumeration

- Legacy interface, introduced in Java 1.0.
- Can only be used with legacy classes: `Vector`, `Stack`, `Properties`.
- Allows forward-only, element-by-element traversal of a group of objects.
- Considered outdated; superseded by `Iterator` in Java 1.2.

Iterator

- Universal cursor: works on any collection (List, Set, Map values, etc.).
- Forward-only traversal (start to end, no reverse).
- Supports read and remove operations (cannot add or replace).
- Obtained by calling `.iterator()` on any collection object.
- Part of the `java.util` package.

```
ArrayList<String> list = new ArrayList<>();
// ... populate list

// Option 1: Enhanced for loop
for (String str : list) {
    System.out.println(str);
}

// Option 2: Index-based loop (only works because ArrayList implements RandomAccess)
for (int i = 0; i < list.size(); i++) {
    String str = list.get(i);
    System.out.println(str);
}

// Option 3: Iterator (most universally applicable)
Iterator<String> objIterator = list.iterator();
while (objIterator.hasNext()) {
    System.out.println(objIterator.next());
}
```

- Iterator drawbacks:
 - Forward-only: cannot traverse backwards.
 - No replacement: cannot update an element in place.
 - No insertion: cannot add new elements during traversal.

ListIterator

- Extends `Iterator` with full bidirectional traversal.
- Supports read, remove, add, and replace operations.
- Obtained via `.listIterator()` on a `List` object.
- Only available for `List` types (not Set or Map), because ListIterator is a List-specific cursor.
- Overcomes all three drawbacks of Iterator.

2. The Iterable Interface and Inner Class Implementation

- For any class to support the enhanced for loop or iterator-based traversal, it must implement the `Iterable` interface.
- `Iterable` declares that the class is capable of generating an `Iterator`.

- Internally, Java collection classes implement this by defining a **private inner class** (e.g., `Itr`) that implements the `Iterator` interface.
- This inner class defines `hasNext()` and `next()` (and `remove()`).
- Because the inner class is private, only the enclosing collection class can instantiate it.
- When you call `list.iterator()`, the method returns a new `Itr()` object of this private inner class.

CONCEPTUAL ILLUSTRATION OF HOW ARRAYLIST IMPLEMENTS ITERATOR INTERNALLY

```
public class ArrayList<E> implements List<E>, Iterable<E> {

    public Iterator<E> iterator() {
        return new Itr(); // returns instance of private inner class
    }

    private class Itr implements Iterator<E> {
        public boolean hasNext() { /* ... */ }
        public E next() { /* ... */ }
        public void remove() { /* ... */ }
    }
}
```

- `ListIterator` similarly has its own inner class that extends `Itr` and adds `previous()`, `nextIndex()`, `previousIndex()`, `set()`, and `add()`.
- If you were to build your own custom collection class and wanted it to support iterator-based traversal, you would need to implement `Iterable` and provide your own inner class implementing `Iterator`.

3. List Implementations: ArrayList, LinkedList, Vector, Stack

ArrayList

- Internally uses a **growable array** as its underlying data structure.
- Implements `RandomAccess`, meaning any element can be accessed directly by index in $O(1)$.
- Allows duplicate values and null values.
- Preserves insertion order.
- Performance concern: insertion and deletion are expensive.
 - To insert or remove an element, a new array must be created and all elements copied over.
 - When the array capacity is exhausted (e.g., 10 elements), a new array of size 11 is created, 10 elements are copied, and the new element is inserted; next add creates size 12, and so on.
- Best used when frequent traversal/retrieval is required, not frequent insert/delete.

LinkedList

- Internally uses a **doubly linked list** as its underlying data structure.
- Bidirectional traversal is inherent to the structure.
- Allows heterogeneous data, duplicate values, and null values.
- Preserves insertion order: elements are stored in the sequence they were added.
- Does **not** implement `RandomAccess`: to reach the 3rd element, you must traverse from the head (or tail).
- Best used when frequent insertion, deletion, or addition is required.
- Not ideal when frequent traversal/retrieval is the primary operation.

Vector

- Also uses a **growable array** internally, similar to `ArrayList`.
- Key difference: `Vector` is **thread-safe** — all its methods are marked `synchronized`.
- Thread safety means: when multiple threads access the same `Vector` object simultaneously, only one thread can operate on it at any given moment, preventing data inconsistency.
- Analogy from lecture: if three students (Rutuj, Puja, Rachna) all try to answer a teacher simultaneously, the answers get jumbled. Synchronization ensures only one speaks at a time.
- Performance trade-off: synchronization overhead makes `Vector` slower than `ArrayList` in single-threaded scenarios.
- Like `ArrayList`, not advisable for frequent insert/delete; best for frequent retrieval.

Stack

- Extends `Vector`, so it is also **thread-safe**.
- Follows **LIFO** (Last In, First Out) principle.
- Unique methods:

```
Stack<Integer> obj = new Stack<>();
obj.push(1);    // adds 1 to top
obj.push(2);    // adds 2 to top
obj.push(3);    // adds 3 to top
// Stack state (top to bottom): 3, 2, 1

obj.pop();      // retrieves AND removes 3 from top; top is now 2
obj.peek();     // retrieves 2 from top WITHOUT removing it; top remains 2
obj.search(1);  // returns offset from top; 1 is at position 2 (0-indexed from top)
```

- `push(obj)` : adds element to the top of the stack.
- `pop()` : retrieves and removes the top element.
- `peek()` : retrieves the top element without removing it.
- `search(obj)` : returns the offset (position from the top, 0-indexed) of the specified object.

4. General Data Structure Terminology

Understanding the underlying data structures helps clarify how Java collection classes behave internally.

Singly Linked List

- Each node holds data and the address of the **next** node.
- The last node points to null, marking the end of the list.
- Traversal is unidirectional: left to right only.

```
[Node1 | next] --> [Node2 | next] --> [Node3 | null]
```

Doubly Linked List

- Each node holds data, the address of the **next** node, and the address of the **previous** node.
- The first node's previous is null; the last node's next is null.
- Traversal is bidirectional.

```
null <-- [prev | Node1 | next] <--> [prev | Node2 | next] <--> [prev | Node3 | next] --> null
```

Circular Linked Lists

- **Circular singly linked list**: the last node points back to the first node instead of null.
- **Circular doubly linked list**: the last node's next points to the first, and the first node's previous points to the last.

Queue (FIFO)

- First In, First Out: the first element inserted is the first to be retrieved and removed.
- Implemented using a linked list where elements are added at the tail and removed from the head.
- Each dequeue operation retrieves the front element, removes it, and shifts the pointer to the next node.

Stack (LIFO)

- Last In, First Out: the most recently inserted element is the first to be retrieved.
- Implemented using a linked list where elements are added and removed from the same end (the top).
- The underlying logic is the same linked list structure; only the retrieval algorithm differs.

The collection framework internally uses these same principles. Whether you use Java's Stack or Queue, the underlying algorithm and data structure logic is what differentiates them — not a fundamentally different implementation.

5. Arrays.asList and Varargs (Variable Argument Lists)

Converting an Array to a List

- The `Arrays` class (a utility class in `java.util`) provides a static method `asList()` that converts an array into a fixed-size List.

```
List<Integer> list = Arrays.asList(40, 90, 78, 33, 1, 66, 11);
```

The Ellipsis (Varargs) Syntax

- The signature of `asList` is: `public static <T> List<T> asList(T... a)`
- The `T... a` syntax is called an **ellipsis** or **variable argument list (varargs)**.
- Originated from the C language (used in `printf` declarations).
- Means the method can accept **any number of arguments** of type T — one, two, three, or more.
- This is **not** function overloading: it is a single method capable of accepting a variable number of parameters.
- The compiler internally treats the varargs parameter as an **array**.

```
// Custom varargs method example
public static void display(Object... data) {
    System.out.println(data[0]); // compiler treats 'data' as an array
}

// All of the following are valid calls to the same single method:
display(1);
display(1, 2);
display(1, 2, 3);
display(1, 2, 3, 4);
```

You can define your own methods using the ellipsis syntax, just as Java's standard library does.

6. Collections Utility Class

- `Collections` (note the 's') is a utility class in `java.util` that provides static methods operating on collection objects.
- Distinct from the `Collection` interface.

Key Methods

```
List<Integer> list = Arrays.asList(40, 90, 78, 33, 1, 66, 11);

// Sort in natural order (ascending for integers, alphabetical for strings)
Collections.sort(list);
// list is now: [1, 33, 40, 66, 78, 90, 11] -> sorted: [1, 11, 33, 40, 66, 78, 90]

// Reverse the list
Collections.reverse(list);
// list is now in descending order

// Get maximum value
System.out.println(Collections.max(list)); // returns 90

// Get minimum value
System.out.println(Collections.min(list)); // returns 1

// Add all elements from one collection to another
Collections.addAll(targetCollection, sourceCollection);

// Binary search (list must be sorted first)
Collections.binarySearch(list, 66);

// Fill a list with a specific value
Collections.fill(list, 0); // sets all elements to 0
```

- Natural sorting order:
 - Integers: ascending (1, 2, 3...)
 - Strings: alphabetical (A, B, C...)
- `Collections.sort()` and `Collections.reverse()` modify the list in place.
- These utility methods are only applicable to collection types; they are not directly available on arrays (use `Arrays.sort()` for

arrays).

7. Comparable and Comparator

When working with custom (user-defined) objects, Java's `Collections.sort()` does not know how to compare them. You must explicitly define the comparison logic.

Comparable Interface

- Implemented **by the object class itself**.
- Declares that objects of this type have a natural ordering.
- Requires overriding the `compareTo(T o)` method.
- Return values: 0 if equal, 1 if current object is greater, -1 if current object is lesser.
- Limitation: only **one** comparison logic can be defined per class.

EMPLOYEE/ENGINEER IMPLEMENTING COMPARABLE — SORTING BY AGE ASCENDING

```
class Engineer extends Employee implements Comparable<Employee> {  
  
    @Override  
    public int compareTo(Employee o) {  
        Engineer check = (Engineer) o;  
        if (this.age == check.age) return 0;  
        else if (this.age > check.age) return 1;  
        else return -1;  
    }  
}
```

USAGE WITH COLLECTIONS.SORT

```
List<Employee> list = new ArrayList<>();  
Employee e1 = new Engineer("Ravi", "Mumbai", 22, 'M', 1.18f, 1.1f);  
Employee e2 = new Engineer("Rita", "Delhi", 33, 'F', 1.5f, 1.2f);  
Employee e3 = new Engineer("Arun", "Pune", 18, 'M', 1.0f, 1.0f);  
Employee e4 = new Engineer("Pooja", "Mumbai", 44, 'F', 2.0f, 1.5f);  
  
list.add(e1);  
list.add(e2);  
list.add(e3);  
list.add(e4);  
  
Collections.sort(list);  
// Output order by age: Arun(18), Ravi(22), Rita(33), Pooja(44)
```

To sort in **descending** order, simply reverse the return values:

```
if (this.age > check.age) return -1;  
else return 1;  
// Output order: Pooja(44), Rita(33), Ravi(22), Arun(18)
```

Type safety: use the generic form `Comparable<Employee>` so the compiler knows the exact type being compared, eliminating warnings.

Comparator Interface

- Implemented by a **separate, independent class** — not the object being compared.
- Separates sorting/comparison business logic from the core object logic.
- Allows **multiple** comparison strategies for the same object type.
- Requires overriding `compare(T o1, T o2)` — both objects are passed as arguments.
- Return values: same convention (0, 1, -1).

COMPARATOR SORTING BY AGE

```
class EmployeeAgeComparator implements Comparator<Employee> {  
    @Override  
    public int compare(Employee o1, Employee o2) {  
        if (o1.age == o2.age) return 0;  
        else if (o1.age > o2.age) return 1;  
        else return -1;  
    }  
}
```

```
}
```

COMPARATOR SORTING BY NAME, WITH AGE AS SECONDARY CRITERIA

```
class EmployeeNameComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee o1, Employee o2) {
        if (o1.getName().equals(o2.getName())) {
            // Secondary criteria: if names are equal, sort by age
            if (o1.getAge() > o2.getAge()) return 1;
            else return -1;
        }
        if (o1.getName().compareTo(o2.getName()) >= 1) return 1;
        else return -1;
    }
}
```

Usage: pass the comparator as the second argument to `Collections.sort()`.

```
// Sort by age
Collections.sort(list, new EmployeeAgeComparator());
// Output: Arun(18), Ravi(22), Rita(33), Pooja(44)

// Sort by name (with age as tiebreaker)
// Change Ravi's age to 18 to demonstrate tiebreaker:
// Ravi(18), Ravi(22), Pooja(44), Rita(33)
// After name sort: Pooja, Ravi(18), Ravi(22), Rita
Collections.sort(list, new EmployeeNameComparator());
```

- Key distinction:
 - Use **Comparable** when the object itself should define its natural ordering and only one ordering is needed.
 - Use **Comparator** when you want to separate sorting logic from the object, or when you need multiple different orderings.

8. Set Interface

- Represents a group of unique objects as a single entity.
- Key rules:
 - **Duplicates not allowed:** calling `add()` with a duplicate silently ignores the entry (no exception); `add()` returns `false` on failed insertion, `true` on success.
 - **Null allowed, but only once:** a second null insertion is treated as a duplicate and ignored.
 - **Insertion order not preserved:** the Set uses its own internal ordering mechanism.
 - No `ListIterator` available (that is a List-only cursor); only `Iterator` is supported.

```
Set<String> obj = new HashSet<>();
obj.add("Ravi");
obj.add("Pooja");
obj.add("Ravi"); // duplicate - silently ignored
obj.add("pooja"); // NOT a duplicate - different case, different object
obj.add(null); // allowed once
obj.add(null); // duplicate null - ignored

System.out.println(obj); // prints: [null, Ravi, Pooja, pooja] in some internal order
```

HashSet

- Introduced in Java 1.2.
- Internally uses a `HashMap` to store elements in **buckets**.
- No guaranteed ordering — the output order is determined by the internal hashing algorithm.
- Fastest for add/remove/contains operations (O(1) average).

```
HashSet<Integer> set = new HashSet<>();
set.add(77);
set.add(22);
set.add(99);
set.add(11);
set.add(77); // duplicate - ignored, add() returns false
set.add(null);
```

```
System.out.println(set); // e.g., [null, 99, 22, 77, 11] — order not guaranteed
```

LinkedHashSet

- Introduced in Java 1.4.
- Extends `HashSet` but maintains a doubly linked list to preserve **insertion order**.
- Slightly slower than `HashSet` due to the overhead of maintaining order.

TreeSet

- Implements `NavigableSet` → `SortedSet` → `Set`.
- Stores elements in **natural sorted order** automatically.
- Does **not** allow null (throws `NullPointerException`).
- Internally uses a **binary tree** structure.

9. TreeSet: Binary Tree Insertion Mechanics

When elements are added to a `TreeSet`, they are inserted using binary tree logic:

1. The first element added becomes the **root**.
2. Each subsequent element is compared to the root:
 - If lesser, it goes to the **left branch**.
 - If greater, it goes to the **right branch**.
3. If a branch already has a node, the comparison repeats at that node.

EXAMPLE FROM LECTURE (INSERTING 55, 22, 77, 33, 19, 25, 21)

```
Insert 55 → root
Insert 22 → 22 < 55 → left of 55
Insert 77 → 77 > 55 → right of 55
Insert 33 → 33 < 55 → left; 33 > 22 → right of 22
Insert 19 → 19 < 55 → left; 19 < 22 → left of 22
Insert 25 → 25 < 55 → left; 25 > 22 → right; 25 < 33 → left of 33
Insert 21 → 21 < 55 → left; 21 < 22 → left; 21 > 19 → right of 19
```

- Because every insertion involves a comparison, **sorting is a necessity**, not optional.
- This means any object stored in a `TreeSet` must be **comparable**.
- If the object does not implement `Comparable`, a `ClassCastException` is thrown at the first `add()` call:

```
ClassCastException: Engineer cannot be cast to java.lang.Comparable
```

Making Objects TreeSet-Compatible

OPTION 1: IMPLEMENT COMPARABLE

```
class Engineer extends Employee implements Comparable<Employee> {
    @Override
    public int compareTo(Employee o) {
        Engineer check = (Engineer) o;
        if (this.age == check.age) return 0;
        else if (this.age > check.age) return 1;
        else return -1;
    }
}

TreeSet<Employee> employeeSet = new TreeSet<>();
employeeSet.add(e1);
employeeSet.add(e2);
employeeSet.add(e3);
employeeSet.add(e4);

Iterator<Employee> itr = employeeSet.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}

// Output sorted by age: Arun(18), Ravi(22), Rita(33), Pooja(44)
```

OPTION 2: PASS A COMPARATOR TO THE TREESSET CONSTRUCTOR

```
TreeSet<Employee> employeeSet = new TreeSet<>(new EmployeeAgeComparator());
employeeSet.add(e1);
employeeSet.add(e2);
employeeSet.add(e3);
employeeSet.add(e4);
// Sorted by age via comparator - Employee class itself need not implement Comparable
```

Note on Duplicate Detection: equals() and hashCode()

- When `compareTo()` returns 0, the `TreeSet` treats the two objects as duplicates and does not add the second one.
- If two objects have the same age but different other fields (e.g., same age, different name), they will still be treated as duplicates unless `equals()` and `hashCode()` are overridden.
- Overriding `equals()` and `hashCode()` allows the `TreeSet` to distinguish objects that are “equal” by the comparator but not truly identical.
- This is an advanced topic not fully covered in this lecture.

10. Map Interface

- Represents a group of objects as **key-value pairs**.
- Even though Map is part of the Java Collection Framework, the `Map` interface does **not** inherit from the `Collection` interface — it has its own separate hierarchy.
- Both key and value must be **objects** (no primitives directly; use wrapper types like Integer, String).

Key Rules

- **Duplicate keys not allowed:** inserting a duplicate key **overwrites** the existing value for that key.
- **Duplicate values allowed.**
- **Null key allowed once:** a second null key overwrites the first.
- **Null values allowed** (multiple).

HashMap Example

```
Map<Integer, String> map = new HashMap<>();
map.put(1000, "first");
map.put(1001, "second");
map.put(1002, "third");
map.put(1003, "fourth");
map.put(1004, "first"); // duplicate value - allowed
map.put(1001, "fifth"); // duplicate key - overwrites "second" with "fifth"
map.put(null, 6); // null key - allowed once
map.put(null, 7); // duplicate null key - overwrites previous null entry

System.out.println(map);
// Output: {null=7, 1000=first, 1001=fifth, 1002=third, 1003=fourth, 1004=first}
```

Retrieving Data from a Map

1. GET A SPECIFIC VALUE BY KEY

```
String data = map.get(1002);
System.out.println(data); // prints: third
```

2. GET ALL VALUES (IGNORING KEYS)

```
Collection<String> allValues = map.values();
Iterator<String> itr = allValues.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

3. GET ALL KEYS

```
Set<Integer> allKeys = map.keySet(); // returns a Set because keys are unique
Iterator<Integer> itr = allKeys.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

```
}
```

4. GET KEY-VALUE PAIRS TOGETHER USING ENTRYSET

- Each entry in the map is represented by the inner interface `Map.Entry<K, V>`.
- `Map.Entry` provides `getKey()` and `getValue()` methods.

```
Set<Map.Entry<Integer, String>> allEntries = map.entrySet();
Iterator<Map.Entry<Integer, String>> itr = allEntries.iterator();
while (itr.hasNext()) {
    Map.Entry<Integer, String> entry = itr.next();
    System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
}
```

Real-World Use Cases for Map

- **Token/queue systems:** a placement drive issues unique token numbers to 5,000 candidates; token number is the key, candidate object is the value; retrieval is instant by token.
- **Bank transaction IDs:** each transaction has a unique ID (key) mapped to transaction details (value); RBI can execute transactions in sequence.
- **Stock exchange order books:** orders placed are mapped by order ID; executed in sequence based on the key.

11. TreeMap and Custom Key Sorting

- `TreeMap` sorts entries by **key** in natural order.
- Hierarchy: `Map` → `SortedMap` → `NavigableMap` → `TreeMap`.
- Mirrors the `TreeSet` hierarchy; same rules apply but for key-value pairs.

TREEMAP WITH INTEGER KEYS — SORTED AUTOMATICALLY BY KEY

```
Map<Integer, String> treeMap = new TreeMap<>();
treeMap.put(1003, "fourth");
treeMap.put(1000, "first");
treeMap.put(1002, "third");
treeMap.put(1001, "second");

System.out.println(treeMap);
// Output: {1000=first, 1001=second, 1002=third, 1003=fourth} — sorted by key
```

Custom Key Objects in TreeMap

If the key is a custom object, it must implement `Comparable`, or a `Comparator` must be provided.

OPTION 1: EMPLOYEEID IMPLEMENTS COMPARABLE

```
class EmployeeId implements Comparable<EmployeeId> {
    int id;

    @Override
    public int compareTo(EmployeeId o) {
        if (this.id == o.id) return 0;
        else if (this.id > o.id) return 1;
        else return -1;
    }
}

TreeMap<EmployeeId, Employee> map = new TreeMap<>();
map.put(new EmployeeId(1000), new Engineer("Tushar", ...));
map.put(new EmployeeId(1004), new Engineer("Zahid", ...));
// Sorted by EmployeeId.id automatically
```

OPTION 2: PROVIDE A COMPARATOR

```
class EmployeeIdComparator implements Comparator<EmployeeId> {
    @Override
    public int compare(EmployeeId o1, EmployeeId o2) {
        if (o1.id == o2.id) return 0;
        else if (o1.id > o2.id) return 1;
        else return -1;
    }
}
```

```
TreeMap<EmployeeId, Employee> map = new TreeMap<>(new EmployeeIdComparator());
```

The Comparable/Comparator pattern applies uniformly across all three parts of the collection framework: List, Set, and Map.

12. Apache Commons Collections and the Apache Software Foundation

- The Java Collection Framework (introduced in Java 1.2) covers the essentials, but third-party libraries extend it significantly.
- The **Apache Software Foundation** (apache.org) is a community-driven open-source organization founded by programmers who came together to build tools, libraries, and products for the Java ecosystem.
- Notable Apache projects:
 - **Apache Tomcat**: the most widely known Java web server; supports HTTP and Java technologies (Servlets, JSP).
 - **Apache Ant**: a build tool for Java projects.
 - **Apache Maven**: the most commonly used Java build and dependency management tool; also part of the Apache Foundation.
- Scale: 216 communities managing 308 projects; approximately 53% (around 269 projects) are Java-related.

Apache Commons Collections

- One of the most popular Apache projects.
- Provides a significantly richer set of collection classes and interfaces than the standard Java Collection Framework.
- Includes advanced Map implementations: `CaseInsensitiveMap`, `CompositeMap`, `FlatMap`, `LinkedMap`, and more.
- Also provides utilities for cryptography, compression, and many other domains.
- Available as a downloadable library with full documentation.
- Apache is not the only such foundation; hundreds of similar libraries exist and can be found via a simple search.

13. Arrays: Single-Dimensional, Matrix, and Jagged

Single-Dimensional Array

Three equivalent declaration syntaxes:

```
// Syntax 1: fixed size, no initial values (all elements default to 0)
int[] sd = new int[10];

// Syntax 2: fixed size with predefined values
int[] sd = new int[] {1, 2, 3, 4, 5};

// Syntax 3: shorthand with predefined values (equivalent to Syntax 2)
int[] sd = {1, 2, 3, 4, 5};
```

Matrix Array (2D, Equal Columns per Row)

- Every row has the **same number of columns**.

```
int[][] matrix = new int[5][10]; // 5 rows, each with 10 columns
// matrix[0] has columns 0 through 9
// matrix[4] has columns 0 through 9
matrix[0][0] = 42; // valid
matrix[4][9] = 99; // valid
```

Jagged Array (2D, Variable Columns per Row)

- Number of rows is defined, but **columns per row are defined separately**.
- Each row can have a different number of columns.

```
int[][] jag = new int[3][]; // 3 rows, columns not yet defined

jag[0] = new int[3]; // row 0 has 3 columns (indices 0, 1, 2)
jag[1] = new int[10]; // row 1 has 10 columns (indices 0 through 9)
jag[2] = new int[5]; // row 2 has 5 columns (indices 0 through 4)

jag[0][0] = 1; // valid
jag[0][2] = 3; // valid
```

```
jag[0][3] = 99; // ArrayIndexOutOfBoundsException! row 0 only has indices 0-2  
  
jag[1][9] = 55; // valid - row 1 has 10 columns
```

- Attempting to access `jag[0][3]` throws `ArrayIndexOutOfBoundsException` because row 0 was defined with only 3 elements (indices 0, 1, 2).
- Jagged arrays are useful when each row genuinely requires a different number of elements, saving memory compared to a full matrix.

14. Upcoming Topics and Next Steps

Exercises Assigned

- Implement ArrayList, Collections utility class, and TreeSet exercises (approximately 9–10 problems).
- Practice implementing Comparable and Comparator on custom objects.
- Explore Arrays class methods for converting arrays to collections and vice versa.
- Read through documentation for each collection class to discover additional methods.

Next Session: Monday, 4:30 PM

- **Lambda expressions** and functional programming in Java.
- After lambda expressions: **Stream API** (filtering, mapping, reducing, sorting collections with streams).

Remaining Major Topics

- Lambda expressions
- Stream API (filter, map, reduce, sort)
- Multithreading and synchronization
- JDBC (Java Database Connectivity)

Smaller Pending Topics (each approximately 1 hour or less)

- Enums (enumeration types)
- Inner classes
- Records (introduced in modern Java)
- Static imports
- Jagged arrays (covered briefly above)

Meeting transcript reference: notes.granola.ai/t/bb3db148...